# Plano de validação — regra do fluxo de trabalho

Data: 2026-09-04. Vale a partir do bloco 4.3 (aplicado retroativamente a ele).

## A regra, em uma frase

**Nenhuma sprint fecha antes de `bash scripts/validacao/validar.sh` passar, e o que o plano não cobre entra como pendência escrita no documento do bloco.**

### Quando roda (desde 2026-10-01, a pedido do Breno)

O plano roda **ao fim de cada bloco `m.n`**, e o bloco só sai como validado se a rodada dele passar. O fechamento da sprint é a rodada do último bloco. Fora disso, roda quando o Breno pedir.

Histórico: até 2026-09-23 já se rodava por bloco (os documentos do 4.3 ao 6.3). De 2026-09-23 a 2026-10-01, só no fechamento da sprint. A regra voltou porque o fechamento da sprint 11 acumulou cinco blocos sem rodar e precisou de sete rodadas para achar os erros do plano escritos bloco a bloco (`docs/sprint-11/11.5-status-entrega.md`).

São **sete camadas**, da mais barata à mais cara: estático, unitário, build, runtime, dados, escrita e navegador.

Comando: `npm run validar` (ou `bash scripts/validacao/validar.sh`). Camada isolada: `bash scripts/validacao/validar.sh escrita`. O que cada uma faz: `bash scripts/validacao/validar.sh --lista`.

## Por que existe

Do 4.1 ao 4.3, a validação foi improvisada no fim de cada bloco e sempre com a mesma lacuna: `tsc`, `lint`, `npm test` e `npm run build` passando, e **nenhuma linha executada contra o banco**. Isso deixa passar a classe de erro mais cara do projeto — coluna que existe no type gerado mas não no banco, JOIN aninhado que o PostgREST resolve diferente do que o tipo promete, policy de RLS que devolve zero linha pro perfil errado. O build não vê nenhum dos três.

O plano fecha essa lacuna com duas camadas que **hoje rodam automatizadas**: runtime (a rota abre, autenticada, num servidor de verdade) e dados (as queries da tela rodam contra gc-dev sob RLS).

Efeito colateral já colhido: na primeira execução, o plano apontou que o documento do 4.3 dizia "22 rotas" quando são 29 — número que eu havia lido de `Generating static pages (23/23)` em vez da tabela de rotas.

## Pasta de build própria

Desde 2026-09-29, o `validar.sh` exporta `NEXT_DIST_DIR=.next-validacao`, e o `next.config.mjs` usa essa pasta como `distDir`. O build, o `next start` do runtime e da escrita e o `next dev` do navegador ficam todos em `.next-validacao/`, ignorada pelo git.

Antes, a validação buildava na mesma `.next` do `next dev` aberto pelo Breno. Com o dev server no ar, as páginas passavam a sair sem CSS e com código velho, porque os arquivos que ele referenciava tinham sido regravados pelo `next build`. Quem roda uma camada fora do `validar.sh` precisa exportar a mesma variável.

## As camadas

Ordem fixa, do mais barato ao mais caro. **Para no primeiro erro**: camada barata que falha invalida as caras, e seguir em frente só produz ruído.

### 1. Estático — `npx tsc --noEmit` + `npm run lint`

| | |
|---|---|
| **Prova** | O código compila e o tipo do `select()` bate com `src/lib/supabase/types.ts`. Com `as` (nunca `as unknown as`), coluna inexistente no type quebra o build. |
| **Não prova** | Que roda. Nem que o type gerado corresponde ao banco de hoje. |
| **Passou** | Saída vazia no `tsc`; "No ESLint warnings or errors". |
| **Falhou** | Qualquer erro. Não existe "warning aceitável" aqui. |
| **Custo** | ~7s |

### 2. Unitário — `npm test`

| | |
|---|---|
| **Prova** | A regra pura: transições de status, os dois CHECKs de `pct_*`, conversão UI↔banco, `computePeriodoCutoff`, `sanitizeBusca`, `obraIdsDaBusca`. |
| **Não prova** | Query, render, integração. Nada que toque banco ou React. |
| **Passou** | `fail 0`. O resumo do `validar.sh` imprime a contagem de casos. |
| **Falhou** | `fail` > 0. |
| **Custo** | ~1s |

`node --test` do Node 24 roda `.ts` direto (type stripping), sem runner instalado — por isso os imports nos testes trazem a extensão `.ts`.

### 3. Build — `npm run build` + diff de rotas

| | |
|---|---|
| **Prova** | Toda rota monta (Server Components, `generateStaticParams`, middleware) e **nenhuma rota sumiu sem intenção**: a lista é comparada com `scripts/validacao/rotas-esperadas.txt`. |
| **Não prova** | Que qualquer página abre. Build passa com página que dá 500 no primeiro acesso. |
| **Passou** | Build sem erro e lista idêntica à baseline (hoje 29 rotas). |
| **Falhou** | Erro de build, ou rota adicionada/removida sem atualizar a baseline. |
| **Custo** | ~40s |

Rota nova é mudança intencional: `bash scripts/validacao/validar.sh build --aceitar-rotas` regrava a baseline. O passo existe pra a atualização ser deliberada, não silenciosa.

### 4. Runtime — `next start` + fetch autenticado

| | |
|---|---|
| **Prova** | Cada rota de `scripts/validacao/validacao-rotas.json`: **sem sessão** redireciona pro `/login` (o guard funciona) e **com sessão real** devolve o status esperado (200, ou o `esperaStatus` de uma rota que redireciona por regra de negócio) com os trechos esperados no HTML. Renderização de Server Component com dados de verdade. |
| **Não prova** | Aparência, layout, comportamento de clique, JS de cliente. Só o HTML do servidor. |
| **Passou** | Todas as rotas ok; o script imprime `N/N rotas ok`. |
| **Falhou** | 200 onde devia redirecionar, 500, ou trecho esperado ausente. |
| **Custo** | ~10s (depende do build já feito) |

**Como a sessão é obtida sem navegador:** o script faz `signInWithPassword` com a anon key e monta o cookie no formato que o `@supabase/ssr` lê — `sb-<ref>-auth-token` = `base64-` + base64url do objeto `session`. O servidor trata a requisição como usuário logado, com RLS ativa. Não é mock: é o mesmo caminho de um navegador.

Rotas novas entram em `scripts/validacao/validacao-rotas.json`, com os trechos de HTML que provam que a tela renderizou o que deveria. Cada bloco acrescenta as suas.

**Toda rota precisa de pelo menos uma asserção positiva**, e o validador recusa `esperaHtml` vazio. Junto vai uma asserção negativa global: nenhuma rota pode conter "Erro ao carregar" nem "Application error". As duas regras existem por um caso concreto — `/obras` passou semanas quebrada com a validação dizendo "ok", porque a rota só conferia o status e a caixa de erro renderiza dentro de um HTTP 200. Ver `docs/tecnicos/correcoes-listagens-e-obras.md`.

Quatro detalhes de quem escreve as asserções:

- **Trecho esperado nunca atravessa interpolação de JSX.** O SSR do React insere `<!-- -->` entre texto literal e `{valor}`, então `Proposta {numero}` não existe como string contínua no HTML. Espere `SEED-VENCIDA-001`, não `Proposta SEED-VENCIDA-001`. Isso já reprovou uma execução com a página certa.
- **Rota dinâmica usa placeholder.** `{propostaSeed}` no path é resolvido em tempo de execução para o id da proposta de `scripts/banco/seed-propostas-dev.mjs` — é assim que `/propostas/[id]` entra sem uuid chumbado no arquivo. Sem o seed, a rota é pulada com instrução, não falha.
- **Redirect por regra de negócio é asserção legítima.** `esperaStatus: 307` em `/propostas/{id}/editar` prova que proposta enviada não é editável.
- **Regra de permissão se prova com outro login.** `"perfil": "visualizador"`, `"comercial"` ou `"financeiro"` numa rota faz o validador entrar com aquele usuário (`VALIDACAO_EMAIL_<PERFIL>` no `.env.local`) — é assim que "visualizador não vê o botão Nova proposta" deixa de ser só uma linha de código. Senha desconhecida se resolve com `scripts/banco/resetar-senha-dev.mjs`, que só aceita `@teste.com` e só roda contra gc-dev.
- **Ausência precisa de contraprova.** Ao afirmar "perfil X não acessa Y", acrescente uma rota que o perfil X **acessa** — sem ela, um erro de sessão passa como sucesso do teste de permissão. É por isso que financeiro, produção e medição têm `/fd` ou `/execucao` na lista, ao lado do `/propostas` que redireciona.
- **Perfil de teste não precisa de senha.** `scripts/comum/sessao-dev.mjs` gera sessão pelo Admin API (`generateLink` + `verifyOtp`) com o service role que o `.env.local` já tem. Perfil novo é **uma linha** em `PERFIS_DE_TESTE`, sem reset de senha nem variável nova — e nenhuma senha de teste fica guardada.
- **Export tem asserção de conteúdo, não só de `content-type`.** `esperaXlsx` numa rota faz o validador abrir a planilha com o `exceljs` e conferir colunas, quantidade de linhas e textos de célula. Um `content-type` correto com colunas trocadas passaria batido.
- **Upload pela tela precisa de `DOM.setFileInputFiles`.** `input.files` é read-only em JS; `anexarArquivo` no driver é o único caminho pra exercitar o seletor de arquivo. E cuidado com o que se afere: o `FileUpload` mostra o nome na fila **antes** de terminar e a mantém **depois** do sucesso, então o sinal confiável é a lista salva ("Nenhum anexo ainda" saindo e voltando), não o texto do nome.

### 5. Dados — queries reais contra gc-dev, sob RLS

| | |
|---|---|
| **Prova** | Que os `select()` da aplicação são aceitos pelo PostgREST, que os JOINs voltam na forma que o tipo promete (objeto, não array), que funções e views chamadas pelas telas executam, e que a RLS entrega linha pro perfil usado. É a camada que faltava. |
| **Não prova** | A tela. Uma query certa alimentando uma coluna errada passa aqui. |
| **Passou** | Nenhum erro do PostgREST e nenhum `valida` reprovando; `N/N checagens ok`. |
| **Falhou** | Erro do PostgREST, ou forma inesperada no retorno. |

| | |
|---|---|
| **Custo** | ~1s |

**Zero linha não é falha, mas é reportado** com "query válida, mas não exercitada" — banco de dev pode estar vazio, e o script não pode inventar cobertura que não teve. Quando um filtro volta vazio, isso vira pendência no doc do bloco (é o caso do `?vencidas=1` no 4.3).

As checagens ficam em `scripts/validacao/validar-dados.mjs`, uma por entidade/tela, com a query **copiada do `page.tsx`** — reescrever por fora valida outra coisa.

**Função chamada por view entra em checagem própria, rodada para todas as linhas.** `calcular_valores_obra` quebrava só na obra que tinha contrato, e como a view a chama via `LATERAL`, essa obra derrubava a listagem inteira. Uma checagem que rodasse só a primeira linha teria passado.

### 6. Escrita — Server Actions chamadas por HTTP

| | |
|---|---|
| **Prova** | O caminho de gravação inteiro: criar, editar, mudar status, gravar histórico, subir e remover anexo, excluir — cada um passando pelo guard de perfil, pelo zod, pelos CHECKs do banco e pela RLS do Storage. Também prova o que **não** pode: número repetido, desconto maior que o total, editar proposta enviada, reabrir proposta terminal, e `visualizador` tentando escrever. |
| **Não prova** | O clique. O formulário da tela, o diálogo de status e o seletor de arquivo não são acionados — o que roda é a action que eles chamariam. |
| **Passou** | `N/N passos ok`, e nada sobra em gc-dev. |
| **Falhou** | Qualquer passo, inclusive a limpeza. |
| **Custo** | ~15s (depois do build) |

**Como chama a action sem navegador:** Server Action é um POST com o header `Next-Action: <id>` e o corpo no formato do React Flight. Os ids saem do próprio build — `.next/server/app/**/page.js` traz cada id ao lado do nome da função exportada —, então o mapa se refaz a cada build e nenhum hash fica chumbado no script.

Duas sutilezas que custaram tentativa até acertar, e que valem pra quem for acrescentar uma action:

1. Argumento simples vai como `JSON.stringify([...args])` em `text/plain`.
2. Com `FormData`, o corpo é multipart e **a parte do arquivo precisa vir antes** do campo `0` — o decodificador lê o `0` e procura partes já vistas. Na ordem inversa a action recebe um FormData vazio e responde "Arquivo ausente no upload", que parece bug da aplicação e não é.

**A camada limpa o que cria.** A proposta e o orçamento de teste nascem com número `VALIDA-ESCRITA-<timestamp>` e são apagados no fim, inclusive quando um passo falha — as exclusões estão em `finally`, e a última asserção confere que não sobrou linha.

**Regra de negócio que não quebra o build entra aqui.** O status de rejeição é `rejeitada` em propostas e `rejeitado` em orçamentos; usar o errado só faz o histórico deixar de gravar o motivo, silenciosamente. A camada tem um passo por entidade que reprova nesse caso — é o tipo de erro que `tsc` não vê e teste unitário sozinho não pega, porque depende do valor real gravado.

### 7. Navegador — Chrome headless por CDP, sobre o `next dev`

| | |
|---|---|
| **Prova** | O que só existe no cliente: login pelo formulário, o zod do react-hook-form barrando submit inválido, o valor final calculado ao vivo, o aviso de soma acima de 100%, a lógica condicional do diálogo de status, o toast, a navegação pós-sucesso, o botão desabilitado por regra e o `ConfirmDialog`. Deixa screenshots — é a conferência visual, virada asserção. |
| **Não prova** | Aparência em si. Um screenshot prova que renderizou, não que está bonito; e nenhuma asserção pega "feio mas funcional". Também não cobre teclado, leitor de tela e outros navegadores. |
| **Passou** | `N/N passos ok`, sem erro de console, e nada sobra em gc-dev. |
| **Falhou** | Qualquer passo. **Olhe o screenshot antes de mexer no código** — costuma ser a asserção que está errada, não a tela. |
| **Custo** | ~35s |

**Sem dependência nova.** Playwright resolveria, mas o projeto já usa o Chrome local em `scripts/docs/docs-pdf.sh` e o Node 24 tem WebSocket embutido — então o driver CDP em `scripts/comum/navegador-cdp.mjs` custa um arquivo e zero pacote. Roda sobre `next dev` (porta `PORTA+1`), porque a camada exercita o cliente e o dev server dá erro legível.

**Para dirigir o app fora deste roteiro** — "mudei X, quero ver X na tela" — use a skill `/run-gc-sistema` (`.claude/skills/run-gc-sistema/`), que aceita comandos pelo stdin (`ir`, `clicar`, `preencher`, `shot`, `medir`) em cima do mesmo driver. Os gotchas de CDP estão documentados lá.

Três coisas que custaram tentativa e ficam de aviso:

1. **O perfil do Chrome guarda cookie entre execuções.** Sem `Network.clearBrowserCookies` no começo, a segunda rodada já está logada e a asserção "sem sessão redireciona pro login" falha sem motivo.
2. **`Emulation.setDeviceMetricsOverride` com `mobile: true` impõe um viewport mínimo (~552px)** e mascara justamente a largura estreita que se quer medir. Use `mobile: false`. E espere dois frames antes de medir, senão o número sai do layout antigo.
3. **Texto repetido precisa de escopo.** "Excluir" existe no header e no `ConfirmDialog`; sem `[role="dialog"] button`, o clique volta pro header e o diálogo nunca confirma.

E um aviso sobre o toast: em `next dev` a primeira compilação de uma rota leva mais que os ~4s de vida do toast, então o de criação já expirou quando a página aparece. Aferir o toast numa navegação já compilada — o de mudança de status, por exemplo.

### Duas coisas que valem para as camadas que escrevem (desde o bloco 13.2)

**A auditoria registra as escritas de teste, e as camadas limpam o que geraram.** O trigger `auditar_mudanca` (migrations 019 e 020) grava um evento em `auditoria_eventos` a cada insert, update e delete, inclusive os do roteiro. Uma rodada gera por volta de 1.300 eventos, 1.000 deles só da carga de 500 itens do 5.8. `scripts/comum/auditoria-limpeza.mjs` apaga, com a chave de serviço, os eventos desde o início da rodada **cujo registro não existe mais**, mais os de `entidade = 'validacao'`. As camadas escrita e navegador chamam essa limpeza no `finally`. Evento de registro que continua existindo fica, seja de quem for, e é isso que torna a limpeza segura com duas rodadas em paralelo. A leitura é **paginada**: o PostgREST corta em 1000 linhas sem avisar, e na primeira versão a limpeza leu exatamente 1000 e deixou 189 para trás.

**Rodadas em paralelo usam portas próprias.** Com duas sessões no mesmo repositório, rode com `VALIDACAO_PORTA=3211 VALIDACAO_PORTA_CDP=9322` (ou qualquer par livre; a escrita usa também `PORTA+2`, para o receptor do n8n). Até o 13.2, `VALIDACAO_PORTA_CDP` subia o Chrome na porta pedida, mas `validar-navegador.mjs` conectava sempre na 9222 e dirigia o Chrome da outra rodada. O perfil do Chrome também era um só. Agora a porta vai para o script, e o perfil é `/tmp/gc-validacao/chrome-profile-<porta>`. O `.next` continua sendo um por diretório: duas rodadas **no mesmo checkout** ainda se atropelam no build. Para isso, use um worktree.

### O que o `validar.sh` garante antes e entre as camadas (desde o fechamento da sprint 9)

O fechamento da sprint 9 perdeu cinco rodadas para ambiente, nenhuma para código. Cada causa virou uma trava:

| CAUSA NA SPRINT 9 | O QUE O PLANO FAZ AGORA |
|---|---|
| Rodada interrompida deixou `VALIDA-ESCRITA-*` em gc-dev; o contrato ocupou a única obra sem contrato e o runtime falhou com "falta dado" | **Camada 0, sobras:** antes da primeira camada que usa o banco, `scripts/validacao/verificar-sobras.mjs` procura `VALIDA-*` e `RUN-*` (os prefixos dos roteiros) em NFs, contratos, propostas, orçamentos e obras, e para se achar. `--apagar` remove, com itens, fotos e a auditoria delas |
| `{obraSemContrato}` era "qualquer obra sem contrato" de gc-dev, que tem três | O runtime **cria a própria obra** (`VALIDA-RT-<ts>`, na empresa do SEED-CT-001) e a apaga no fim |
| A escrita subia o `next start` com o do runtime ainda na 3111: `EADDRINUSE`, e a escrita testava o servidor que estivesse lá | `derrubar_servidor` no fim do runtime e da escrita, esperando a porta soltar; `exigir_porta_livre` antes de subir qualquer servidor ou Chrome, com o PID de quem ocupa. No fim, só as portas que a rodada abriu são derrubadas |
| O Chrome de quem desenvolve ocupa a 9222; o Chrome da validação abria só em `[::1]` e o script lia o do outro | `VALIDACAO_PORTA_CDP` padrão **9333**, e a porta é conferida antes |
| `fetch failed` com o Supabase no meio da escrita (rede IPv6 com NAT64) | `NODE_OPTIONS=--dns-result-order=ipv4first` exportado para scripts e servidores; login e `generateLink` repetem até 3 vezes **só em erro de rede** (`scripts/comum/rede.mjs`). As actions testadas nunca repetem |

**Envio de documento pela tela.** Sem `VALIDACAO_ENVIO_REAL=1`, a escrita sobe `scripts/validacao/receptor-n8n.mjs` na porta `PORTA+2` e aponta `N8N_DOCUMENTO_WEBHOOK_URL` (com um token de teste) para ele. O roteiro confere o que chegou: o token no `x-documento-token`, JSON, canal `SISTEMA`, empresa, obra, nome do arquivo e a URL assinada, que baixa o PDF sem sessão. Isso prova o que o app manda; **não prova o n8n** (leitura e gravação de volta), que continua na trilha de automação. Com `VALIDACAO_ENVIO_REAL=1` o envio vai ao n8n do `.env.local`, como antes.

**Seed da automação.** `supabase/seed_automacao.sql` (o SEED-DOC-001, com andamento NA_FILA → CONCLUIDO de 10 dias atrás) exercita as duas queries da automação que saíam "não exercitadas". Elas agora têm `exigeSeed`: com zero linha, a camada dados falha e diz qual seed aplicar.

## Fumaça e CI (13.5)

`bash scripts/validacao/validar.sh fumaca` roda, fora da rodada completa, a camada build e os quatro caminhos críticos por HTTP contra o gc-dev: login e redirecionamento por perfil; orçamento → proposta → contrato → item; a cascata da execução; e NF → pagamento → status. São 22 passos em cerca de 1min30 (`scripts/validacao/validar-fumaca.mjs`). Ela **não substitui** o plano: é o atalho de quando não dá tempo das sete camadas, e o modo padrão do botão do CI. Usa o mesmo mecanismo da camada escrita (`scripts/validacao/actions-http.mjs`), apaga o que cria e marca tudo com `VALIDA-FUMACA-*`, o que o `verificar-sobras.mjs` acha.

O CI (`.github/workflows/validar.yml`) roda estático, unitário e build em todo push. Contra o gc-dev, roda pelo botão "Run workflow", fumaça ou completo, só no repositório com a variável `VALIDAR_GC_DEV=sim`. Detalhes em `docs/sprint-13/13.5-status-entrega.md`.

## O que continua manual

Duas coisas, deliberadamente. Não há automação disso hoje, e finge-la seria pior que a ausência:

1. **Julgamento visual.** A camada 7 clica, mede largura e guarda screenshot — mas quem decide se o resultado está aceitável é uma pessoa olhando. Ela provou que a tabela rola no próprio container em 768px, e mediu que em 390px sobram 150px de conteúdo; **que isso seja inaceitável é conclusão humana**, não asserção. **Passo mínimo:** ao mexer em layout, olhar os screenshots que a camada deixa em `/tmp/gc-validacao/shots`.
2. ~~Escrita no banco.~~ **Deixou de ser manual em 2026-09-05**, com a camada 6. O que sobrou de manual é menor e mais específico: **o clique**. Nenhum formulário é preenchido, nenhum diálogo é aberto, nenhum arquivo é escolhido no seletor — a camada chama a action que a tela chamaria. Fica de fora, então, o que só o navegador exercita: validação do zod no cliente, estado do react-hook-form, o toast, a navegação depois do sucesso.

   **Passo mínimo:** ao mexer num formulário ou diálogo, submeter uma vez pela tela em gc-dev e registrar no doc do bloco. Ao mexer só na action, a camada 6 basta.

Duas coisas que **poderiam** ser automatizadas e não foram, com o porquê:

- **Multiperfil.** Hoje a validação loga com um usuário só (admin). O `setup_inicial_dev.sql` cria seis perfis, então dá pra validar que `visualizador` não vê o botão "Nova proposta" e que `producao` é redirecionado. Fica pro bloco que mexer em permissão — a estrutura de `scripts/validacao/validacao-rotas.json` já tem o campo `perfil` reservado.
- **Schema vs. types gerados.** `supabase/verificar_schema_aplicado.sql` existe e é somente leitura, mas depende de `supabase link` e do personal access token, que é a pendência herdada do 4.1. Enquanto isso, a camada dados cobre a parte que importa na prática: se a coluna não existe mais, a query falha.

## Onde a regra está escrita

| LUGAR | O QUE DIZ |
|---|---|
| `CLAUDE.md` | a regra operacional, lida automaticamente em toda sessão |
| memória do projeto | `plano-de-validacao.md`, pra sobreviver a `/clear` |
| este documento | o plano completo, com o que cada camada prova e não prova |
| `package.json` | `npm run validar` |

## Como fica o doc de cada bloco

A seção "Verificação" passa a ter as cinco camadas com **números reais**, não "passa" genérico:

| CAMADA | RESULTADO |
|---|---|
| estático | `tsc` limpo, lint sem warnings |
| unitário | 46 casos, `fail 0` |
| build | 31 rotas, iguais à baseline |
| runtime | 23/23 rotas, 4 delas como visualizador |
| dados | 12/12 checagens sob RLS |
| manual | telas conferidas no navegador: `/propostas` (desktop e 390px) |

E logo abaixo, **o que não foi validado**, nominalmente. Camada que não rodou aparece como "não rodou", nunca omitida.

## Execução de referência (2026-09-04, bloco 4.3)

```
>> 1/5 estático — tsc + lint          ok (7s)
>> 2/5 unitário — npm test            ok (1s, 31 casos)
>> 3/5 build — next build + rotas     ok (41s, 29 rotas)
>> 4/5 runtime — next start + fetch   ok (10s, 9/9 rotas)
>> 5/5 dados — queries sob RLS        ok (1s, 6/6 checagens, 1 sem registro)
```

Os tempos são de execução em primeiro plano. Rodando em background o processo pode ser suspenso pelo sistema entre as camadas, e aí o número impresso é wall-clock inflado (numa execução assim, o build apareceu como 7340s) — não é regressão de performance.

Achados desta primeira execução, ambos corrigidos: a contagem de rotas do doc do 4.3 estava errada (22 → 29), e a pendência "listagem não verificada contra o banco" foi fechada — `/propostas` devolve 200 autenticado, com `<table>` e as sete colunas, e o JOIN aninhado `obra → cliente` volta objeto com uma proposta real de gc-dev.

## Arquivos

| ARQUIVO | O QUE É |
|---|---|
| `scripts/validacao/validar.sh` | orquestrador das cinco camadas; `--lista`, camada isolada, `--aceitar-rotas` |
| `scripts/validacao/validar-runtime.mjs` | camada 4: sessão real + fetch das rotas |
| `scripts/validacao/validar-dados.mjs` | camada 5: queries da aplicação sob RLS |
| `scripts/validacao/validar-escrita.mjs` | camada 6: Server Actions por HTTP, com limpeza |
| `scripts/validacao/validar-navegador.mjs` | camada 7: o roteiro clicado no Chrome |
| `scripts/comum/navegador-cdp.mjs` | driver CDP sem dependência nova |
| `.claude/skills/run-gc-sistema/` | skill `/run-gc-sistema`: dirige o app ad-hoc, fora do roteiro fixo |
| `scripts/comum/sessao-dev.mjs` | sessão de perfil de teste sem senha, via service role |
| `scripts/banco/aplicar-seed.sh` | `npm run seed`: aplica um `supabase/seed_*.sql` em gc-dev |
| `scripts/comum/gc-dev-guard.mjs` | trava de ambiente: nada roda fora de gc-dev |
| `scripts/validacao/validacao-rotas.json` | rotas e trechos de HTML esperados — cada bloco acrescenta |

| ARQUIVO | O QUE É |
| --- | --- |
| `scripts/validacao/rotas-esperadas.txt` | baseline das 29 rotas do build |
| `.env.local` | `VALIDACAO_EMAIL` / `VALIDACAO_SENHA` (gitignored) |